R Functions Lab (Class 06)

Barry Grant

2026-04-16

Background

In this session you will work through the process of developing your own R functions for generating random DNA and protein sequences. You will then use one of your functions to generate a set of short peptides that can be queried against the NCBI non-redundant (**nr**) protein database to ask an interesting biological question: *can we generate short peptide sequences that have never been observed in nature before and what implications might this have for immune system surveillance?*

The process will involve starting slowly with small defined code snippets (where you already know what the answer should be), and then building up to string them together into reusable functions. Finally, you will refine and use these functions to generate biological data of your own and interpret the results.

We will walk through the first two functions (`add()` and `generate_dna()`) together in class. You will then work on the remaining tasks on your own (or with your neighbor). As always, if you have questions please ask in person or on Piazza. The step wise process of developing these skills is important and we want to support you in attaining them as best we can!

Hints

Useful base R functions that you may (or may not) find useful for this lab include: `sample()`, `paste()`, `paste0()`, `if()`, `for()`, `sapply()`, `nchar()`, and `cat()`. Remember you can look up the help page for any function by prefixing it with a question mark (e.g. `?sample`).

The one-letter code for the 20 standard amino acids is:

```
"A", "R", "N", "D", "C", "E", "Q", "G", "H", "I", "L", "K", "M", "F", "P", "S",
"T", "W", "Y", "V"
```

Your Tasks

- Create a new **RStudio project** for this class session. Save it inside the folder where you have been organizing your work to date for this class. It is good practice not to use spaces in your file or folder names (we will see why in our later `UNIX` class).

- Create a new **Quarto document** (or **Rmarkdown document** if you do not have Quarto available) for storing your notes and code for this session. You will **Render** this to a PDF lab report for later submission on gradescope.
- Answer each of the following questions. Make sure your final document can be rendered without errors.

Q1. Write your first R function: `add()`

Write a first simple function `add()` to add numbers together. Make sure to include comments in your function explaining *what* it does and *why* you do things the way you do.

a. Your first version of the function should add two input numbers together. For example, `add(4, 7)` should return `11`. [**1 pt**]

b. For you second version, adapt your first function so it can take a single input vector or two inputs as before. For example, `add(4, 7)` and `add( c(4,7,10) )`. [**1 pt**]

c. Finally, on your own (outside of class) create a third version of your function that can add any number of inputs that the user provides. For example, `add(1, 2, 3, -4)` should return `2`. *Hint:* explore the `...` (dots) argument or the base R function `sum()`. [**2 pts**]

```
# Example of the expected behavior for your final function
add(4, 7)           # should return 11
add( c(4,7,10) )    # should return 21
add(1, 2, 3, -4)    # should return 2
```

Q2. Write a `generate_dna()` function

Write a function `generate_dna()` that returns a random DNA sequence of a length specified by the user.

a. Your first version should return a multi-element vector of single character nucleotides. For example `generate_dna(6)` might return `"A", "T", "T", "G", "A", "C"`. [**1 pt**]

b. Your second version should optionally be able to return either a multi-element vector of single character nucleotides (as before) or a **single character string** (not a vector of single letters but a singe vector of multiple letters). For example `"AAGGCTG"`. [**1 pt**]

c. Finally, create a final version of your function that prints out a FASTA format sequence with an id line indicating the sequence length. For example:

```
>len9
CGAAGGCTG
```

Be sure to include clear comments that explain each step of your function. [**2 pts**]

```
# Example of the expected behavior
generate_dna(11)   # e.g. "ATTTAAGGCTG"
```

Q3. Write a `generate_protein()` function

Write a function `generate_protein()` that returns a random peptide/protein sequence of a length specified by the user. For example `generate_protein(6)` might return `"WQRTAG"`. Your function should:

- Use the single-letter code for all **20 standard amino acids** and no other letters (see earlier list at the beginning of this handout)and include clear comments that explain each step of your function. **[2 pts]**

```
# Example of the expected behavior
generate_protein(6)   # e.g. "WQRTAG"
```

Q4. Generate random protein sequences of length 6 to 13

Adapt and use your `generate_protein()` function to generate a series of random protein sequences ranging from **6 to 13 amino acids** in length (one sequence per length). Take advantage of the base R function `for()` or `sapply()` so that you do not have to call `generate_protein()` eight times by hand.

Be sure to format your results in **FASTA format** ready to paste or upload as a query to NCBI BLASTp. For example:

```
>id.6
NTYHEC
>id.7
MVRSIAW
>id.8
...
```

Hint: the function `cat()` combined with `paste()` and the newline character `"\n"` makes this relatively straightforward. **[2 pts]**

```
# Expected output (the exact sequences will differ each run)
# >id.6
# NTYHEC
# >id.7
# MVRSIAW
# ...
# >id.13
# KLMQPWRTYAHGV
```

Q5. BLASTp search against nr — are your peptides "unique in nature"?

Take your FASTA-formatted peptides from Q4 and run them as a single BLASTp search against the **Non-redundant protein sequences (nr)** database at https://blast.ncbi.nlm.nih.gov/. For this question **do not** restrict the organism (leave the *Organism* field blank so that the full nr database is searched).

For each of your 8 peptides (lengths 6 through 13), inspect the top hit and record:

- the **% identity** of the best hit, and
- the **% query coverage** of the best hit.

We will define a peptide as **"unique in nature"** (in the specific sense used in this lab) if **no match exists with 100% coverage AND 100% identity** to a sequence already in nr. Note that this is a conservative definition — a short exact match may still exist as a sub-string of a longer protein, and will typically be reported by BLASTp with 100% identity but less than 100% coverage of the *subject* (which we are not using here).

In your quarto report create a fill in table along the lines of the following: **[2 pts]**

Length (aa)	Best hit % identity	Best hit % coverage	Unique? (Y/N)
6			
7			
8			
9			
10			
11			
12			
13			

Then answer the following in **1–3 sentences each**:

a. At which sequence length do your randomly generated peptides start to look "unique in nature" (i.e. no 100% coverage + 100% identity hit)? **[1 pt]**

b. Speculate **why** very short random peptides are almost always found in nr, while longer ones typically are not. Your answer should refer both to the size of the sequence space (20^L for a peptide of length L) and to the size of the known protein universe. **[1 pt]**

Q6. Connecting your findings to immunology (MHC class II and T-cell activation)

Your results from Q5 should show an interesting pattern: very short random peptides almost always match something in the nr database, while as the peptides get longer they rapidly start looking **"unique in nature"**. It turns out that biology exploits exactly this transition.

The adaptive immune system needs to distinguish **self** from **non-self** (e.g. a viral or bacterial protein). To do this, **MHC class II molecules** on the surface of antigen-presenting cells (macrophages, dendritic cells, B cells) bind short peptides derived from proteins that have been taken up and degraded inside the cell, and *display* (present) these peptides to **CD4+ T helper cells**. If a T-cell receptor recognizes the displayed peptide as *foreign*, that T cell is activated and an immune response against the invading pathogen begins.

A key biological observation is that MHC class II molecules preferentially bind peptides that are a certain minimum length (peptides shorter than this are not used for presentation).

In **3–6 sentences total** and using your Q5 data and the reasoning from Q5b, what do you think this minimum length is and why might it be a *bad* design choice for the immune system to present very short peptides? **[3 pt]**

Submission

Make sure you **save your Quarto document** and click the **Render** (or Rmarkdown **Knit**) button to generate a PDF-format report without errors. Finally, submit your PDF to gradescope; make sure to indicate on which page your answers to individual questions (Q1a, Q1b, Q2, etc.) can be found.